B.Tech AI & ML

DATA STRUCTURES

50-Page Detailed Exam Notes

Arrays • Linked Lists • Stack • Queue • Trees • Graphs • Searching • Sorting • Hashing

Purpose: Simple language + exam-ready definitions + algorithms + examples + complexity + important questions.

CONTENTS

Unit	Topics
Unit 1	INTRODUCTION TO DATA STRUCTURES
Unit 2	ARRAYS AND LINKED LISTS
Unit 3	STACK AND QUEUE
Unit 4	TREES AND GRAPHS
Unit 5	SEARCHING AND SORTING
Unit 6	HASHING

Exam strategy: For 5/10-mark answers, write definition → diagram/steps → working → complexity → applications → conclusion.

UNIT 1 – INTRODUCTION TO DATA STRUCTURES

1.1 Data and Information

Data means raw facts or values that can be processed by a computer. Examples are 25, “Mahi”, 78.5 and a student ID.
Information is processed data that has meaning. For example, marks entered for students become information when we calculate the average or result.
A computer program needs suitable methods to store and organize data so that it can process it efficiently.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

1.2 What is a Data Structure?

A **data structure** is a particular way of organizing and storing data in memory so that it can be accessed and modified efficiently.
Examples: Array, Linked List, Stack, Queue, Tree, Graph and Hash Table.
The choice of data structure depends on the operations required, such as searching, insertion, deletion or sorting.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 1 – INTRODUCTION TO DATA STRUCTURES

1.3 Need of Data Structures

Data structures are important because large amounts of data cannot be handled efficiently without proper organization.
Main advantages: efficient data access, better memory utilization, easier data management, faster algorithms, and reusable program design.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

1.4 Classification of Data Structures

Data structures can be broadly divided into **primitive** and **non-primitive** structures.
Primitive examples: integer, character, float and boolean.
Non-primitive structures are further divided into **linear** and **non-linear** structures.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 1 – INTRODUCTION TO DATA STRUCTURES

1.5 Linear Data Structures

In a linear data structure, elements are arranged in a sequential order. Except for special cases, each element has a logical predecessor and successor.
Examples: **Array, Linked List, Stack and Queue.**

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

1.6 Non-Linear Data Structures

In non-linear structures, data is not arranged in a single sequence. One element can be connected to several other elements.
Examples: **Tree and Graph.** Trees represent hierarchy, while graphs represent networks or relationships.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 1 – INTRODUCTION TO DATA STRUCTURES

1.7 Static and Dynamic Structures

A **static** structure generally has a size decided before or during creation and is not easily resized. Arrays are the common example.
A **dynamic** structure can grow or shrink during execution. Linked lists are a common example.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

1.8 Abstract Data Type (ADT)

An **ADT** describes what data is stored and what operations are allowed without specifying the internal implementation.
For a Stack ADT, operations may be **push, pop and peek**. The stack can be implemented using an array or linked list.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 1 – INTRODUCTION TO DATA STRUCTURES

1.9 Algorithm

An **algorithm** is a finite sequence of clear and unambiguous steps used to solve a problem.
A good algorithm should have defined input, output, finiteness, correctness and effective steps.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 1 – INTRODUCTION TO DATA STRUCTURES

1.10 Time and Space Complexity

Time complexity describes how the running time of an algorithm grows with input size.
Space complexity describes the memory required by an algorithm, especially additional memory beyond the input.
Common growth rates include O(1), O(log n), O(n), O(n log n) and $O(n^2)$.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

1.11 Big-O Notation

Big-O notation is commonly used to describe an upper-bound growth rate of an algorithm.
O(1): constant time. **O(log n):** logarithmic. **O(n):** linear. **O(n log n):** common in efficient comparison sorting. **$O(n^2)$:** quadratic.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 1 – INTRODUCTION TO DATA STRUCTURES

1.12 Quick Revision

Remember: **Data Structure** → **Organization of data** → **Operations** → **Algorithm** → **Complexity.**
For exams, always write definition, classification, examples, advantages and one practical application.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 2 – ARRAYS AND LINKED LISTS

2.1 Array

An **array** is a collection of elements stored in an indexed sequence. In many programming languages, elements are accessed using an index beginning at 0.
Example: A = [10, 20, 30, 40]. A[2] gives 30.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 2 – ARRAYS AND LINKED LISTS

2.2 Characteristics of Arrays

Important characteristics include indexed access, ordered elements and efficient traversal. Traditional fixed-size arrays occupy a contiguous memory region.
Direct access by index is typically O(1).

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

2.3 Array Traversal

Traversal means visiting every element of an array exactly as required by the algorithm.
Example: for each element x in A, print x. Traversal of n elements takes O(n) time.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 2 – ARRAYS AND LINKED LISTS

2.4 Insertion in Array

To insert an element at a position in the middle, elements after that position may need to be shifted one position to the right.
Therefore insertion at an arbitrary position can take O(n) time in the worst case.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

2.5 Deletion in Array

When an element is deleted from the middle, later elements may need to shift left to fill the gap. Worst-case time is O(n).

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 2 – ARRAYS AND LINKED LISTS

2.6 Searching in Array

Linear search checks elements one by one and works even when data is unsorted. Its worst-case time is O(n). Binary search is faster but requires sorted data; its worst-case time is O(log n).

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 2 – ARRAYS AND LINKED LISTS

2.7 Advantages and Limitations of Arrays

Advantages: simple representation, fast indexed access and good locality of reference.
Limitations: fixed-size arrays may need resizing, and middle insertion/deletion can be expensive.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

2.8 Linked List

A **linked list** consists of nodes. A node contains data and one or more links to other nodes.
Unlike an array, nodes do not need to occupy consecutive memory locations.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 2 – ARRAYS AND LINKED LISTS

2.9 Singly Linked List

A singly linked list node contains **data + next**. The next field stores the reference to the next node. The last node points to NULL/None.
Traversal is normally from the first node toward the end.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

2.10 Doubly Linked List

A doubly linked list node contains **previous link + data + next link**. It supports traversal in both directions.
It requires extra memory for the previous link.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 2 – ARRAYS AND LINKED LISTS

2.11 Circular Linked List

In a circular linked list, the last node points back to the first node. There is no NULL link at the end of the circular chain.
It is useful for cyclic scheduling and repeated traversal.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

2.12 Insertion in Linked List

Insertion can be performed at the beginning, end or a specified position. Once the required node/reference is known, changing links can be efficient.
At the beginning of a linked list, insertion is typically O(1).

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 2 – ARRAYS AND LINKED LISTS

2.13 Deletion in Linked List

Deletion removes a node and reconnects neighboring links. In a singly linked list, the previous node is generally required when deleting a node from the middle.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 2 – ARRAYS AND LINKED LISTS

2.14 Array vs Linked List

Array: indexed access is fast; insertion/deletion in the middle may require shifting.
Linked list: sequential access; insertion/deletion can be efficient after locating the position; extra memory is needed for links.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

2.15 Exam Practice

Short questions: Define array. What is a linked list? Differentiate singly and doubly linked lists.
Long questions: Explain insertion/deletion in arrays and linked lists with diagrams and complexity.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 3 – STACK AND QUEUE

3.1 Stack

A **stack** is a linear data structure that follows **LIFO – Last In, First Out**. The most recently inserted element is removed first.
Example: a stack of plates.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

3.2 Stack Operations

Push: insert an element. **Pop:** remove the top element. **Peek/Top:** read the top element without removing it. With an appropriate implementation, these operations can be O(1).

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 3 – STACK AND QUEUE

3.3 Stack Overflow and Underflow

Overflow occurs when a push is attempted on a full fixed-size stack.
Underflow occurs when pop or another removal operation is attempted on an empty stack.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

3.4 Stack Using Array

A stack can be represented using an array and a variable called **top**. Initially top is often -1. On push, top is increased and the item is stored. On pop, the item at top is removed and top is decreased.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 3 – STACK AND QUEUE

3.5 Applications of Stack

Stacks are used in function calls, recursion, undo operations, browser history, expression conversion/evaluation, parentheses matching and backtracking.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

3.6 Infix, Prefix and Postfix

Infix: A + B. **Prefix:** + A B. **Postfix:** A B +.
Postfix and prefix forms can reduce the need for parentheses and are useful in expression processing.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 3 – STACK AND QUEUE

3.7 Queue

A **queue** is a linear data structure that follows **FIFO – First In, First Out**. The first inserted element is removed first.
Example: people waiting in a line.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 3 – STACK AND QUEUE

3.8 Queue Operations

Enqueue: insert at rear. **Dequeue:** remove from front. **Front/Peek:** inspect the front element.
A queue can be implemented using arrays or linked lists.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

3.9 Circular Queue

A circular queue treats the array as circular. After the last index, the next position is the first index.
This avoids wasting positions created by earlier dequeue operations in a simple linear array implementation.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 3 – STACK AND QUEUE

3.10 Deque

A **Deque** (double-ended queue) allows insertion and deletion at both the front and rear. It can be input-restricted or output-restricted depending on which operations are limited.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

3.11 Priority Queue

In a priority queue, each element has a priority. Removal is based on priority rather than only arrival time. Heaps are commonly used for efficient priority queues.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 3 – STACK AND QUEUE

3.12 Stack vs Queue

Stack: LIFO, insertion/removal at the top.
Queue: FIFO, insertion at rear and removal at front.
Both are linear data structures and have many scheduling or processing applications.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

3.13 Applications of Queue

Queues are used in CPU scheduling, printer spooling, network buffering, customer-service systems and breadth-first search.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 3 – STACK AND QUEUE

3.14 Exam Practice

Practice: define stack and queue; explain push/pop and enqueue/dequeue; write stack applications; explain circular queue with a diagram; differentiate stack and queue.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 4 – TREES AND GRAPHS

4.1 Tree

A **tree** is a non-linear hierarchical data structure consisting of nodes connected by edges. The topmost node is called the root.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

4.2 Tree Terminology

Root: node with no parent. **Parent:** immediate predecessor. **Child:** immediate successor. **Leaf:** node with no children. **Degree:** number of children. **Height:** length of the longest downward path under the chosen convention.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 4 – TREES AND GRAPHS

4.3 Binary Tree

A **binary tree** is a tree in which each node has at most two children, called left and right child.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

4.4 Binary Search Tree

A **BST** maintains an ordering property: keys smaller than a node are placed in the left subtree and larger keys in the right subtree, depending on the duplicate-handling rule.
Search, insertion and deletion can be efficient when the tree remains reasonably balanced.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 4 – TREES AND GRAPHS

4.5 Tree Traversals

Preorder: Root → Left → Right.
Inorder: Left → Root → Right.
Postorder: Left → Right → Root.
For a BST, inorder traversal produces keys in sorted order when the BST ordering is valid.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

4.6 Level Order Traversal

Level order visits nodes level by level, starting from the root. A queue is commonly used to implement it.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 4 – TREES AND GRAPHS

4.7 Heap

A **heap** is commonly represented as a complete binary tree with a heap-order property.
In a **max-heap**, a parent is at least as large as its children. In a **min-heap**, a parent is no larger than its children.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

4.8 Graph

A **graph** consists of vertices (nodes) and edges connecting them. Graphs can represent roads, communication networks, social relationships and dependencies.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 4 – TREES AND GRAPHS

4.9 Directed and Undirected Graph

In an **undirected graph**, an edge has no direction. In a **directed graph**, an edge goes from one vertex to another and direction matters.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 4 – TREES AND GRAPHS

4.10 Weighted Graph

A weighted graph assigns a value to an edge, such as distance, cost, time or capacity.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

4.11 Graph Representation

Adjacency matrix: uses a matrix to record whether an edge exists and may store its weight.
Adjacency list: stores a list of neighboring vertices for each vertex and is often space-efficient for sparse graphs.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 4 – TREES AND GRAPHS

4.12 BFS

Breadth-First Search explores a graph level by level. It uses a queue and a visited mechanism to avoid repeated processing. In an unweighted graph, BFS can find shortest path distances in terms of number of edges.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

4.13 DFS

Depth-First Search explores one path deeply before backtracking. It can be implemented using recursion or an explicit stack.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 4 – TREES AND GRAPHS

4.14 Tree vs Graph

A tree is a connected acyclic structure under the usual definition. A general graph may contain cycles and may be disconnected. A tree with n vertices has n−1 edges.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

4.15 Exam Practice

Practice tree terminology, all four major traversal ideas, BST, heap, graph representations, BFS and DFS. Draw diagrams whenever possible.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 5 – SEARCHING AND SORTING

5.1 Searching

Searching is the process of finding whether a target item exists in a collection and, if required, locating its position.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

5.2 Linear Search

Linear search checks each element from beginning to end until the target is found or the collection ends.
Best case: O(1). Worst case: O(n). It does not require sorted data.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 5 – SEARCHING AND SORTING

5.3 Binary Search

Binary search works on sorted data. It compares the target with the middle element and discards half of the search interval each time.
Worst-case time: **O(log n)**.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 5 – SEARCHING AND SORTING

5.4 Binary Search Steps

1. Set low and high boundaries.
2. Calculate middle.
3. Compare target with middle value.
4. If equal, return the result.
5. If target is smaller, search left half; otherwise search right half.
6. Repeat until found or interval becomes empty.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

5.5 Sorting

Sorting arranges records according to a key, such as ascending numerical value or alphabetical order. Sorting can make searching and reporting easier.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 5 – SEARCHING AND SORTING

5.6 Bubble Sort

Bubble sort repeatedly compares adjacent elements and swaps them when they are in the wrong order. Worst-case time is $O(n^2)$. It is simple but generally inefficient for large datasets.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

5.7 Selection Sort

Selection sort repeatedly finds the minimum element from the unsorted portion and places it at the next position. Time complexity is $O(n^2)$ in typical analysis.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 5 – SEARCHING AND SORTING

5.8 Insertion Sort

Insertion sort builds a sorted portion one element at a time by inserting the next item into its proper position. Worst-case time is $O(n^2)$, but it can perform well on small or nearly sorted data.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

5.9 Merge Sort

Merge sort uses divide and conquer: divide the list, recursively sort the parts, then merge them.
Typical time complexity is $O(n \log n)$. It usually needs additional memory for merging.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 5 – SEARCHING AND SORTING

5.10 Quick Sort

Quick sort chooses a pivot, partitions the array around it, then recursively sorts the partitions.
Average time is O(n log n); worst case can be $O(n^2)$, depending on pivot choices and partition behavior.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 5 – SEARCHING AND SORTING

5.11 Stable Sorting

A stable sorting algorithm preserves the relative order of records with equal keys. Stability is useful when sorting records by multiple fields in stages.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

5.12 Comparison Table

Linear Search: unsorted data, O(n) worst case.
Binary Search: sorted data, O(log n) worst case.
Bubble/Selection/Insertion: generally $O(n^2)$ worst case.
Merge Sort: O(n log n).
Quick Sort: average O(n log n), worst $O(n^2)$.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 5 – SEARCHING AND SORTING

5.13 Exam Practice

Write algorithms/pseudocode for linear search and binary search. Explain bubble, selection, insertion, merge and quick sort with a small example and complexity.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 6 – HASHING

6.1 Hashing

Hashing is a technique that maps a key to a table position using a hash function. It is widely used for fast average-case insertion, search and deletion.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 6 – HASHING

6.2 Hash Table

A **hash table** is an array-like structure where records are stored at positions calculated from their keys. It can store key-value pairs or records.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

6.3 Hash Function

A **hash function** converts a key into an index in the table. A useful hash function should be fast and should distribute keys reasonably uniformly.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 6 – HASHING

6.4 Collision

A **collision** occurs when two different keys produce the same hash-table index. Since collisions are unavoidable in many practical settings, collision-resolution techniques are required.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

6.5 Separate Chaining

In **separate chaining**, each table position maintains a collection of entries, often represented by a linked list. Multiple keys can therefore share the same index.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 6 – HASHING

6.6 Linear Probing

Linear probing is an open-addressing technique. After a collision, the table checks the next positions sequentially until a suitable position is found.
It is simple but can suffer from primary clustering.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 6 – HASHING

6.7 Quadratic Probing

Quadratic probing checks positions using increasing quadratic offsets from the original hash index. It can reduce some clustering compared with linear probing.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

6.8 Double Hashing

Double hashing uses a second hash function to determine the probe step. It can provide better distribution than simple linear probing when designed correctly.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 6 – HASHING

6.9 Load Factor

Load factor is commonly expressed as α **= number of stored entries / table capacity**.
As load factor increases, collisions may become more frequent, so resizing or rehashing may be needed.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

6.10 Hashing Complexity

With a good hash function and controlled load factor, average search/insertion/deletion can be close to **O(1)**.
Worst-case performance can degrade to O(n) when many keys collide.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 6 – HASHING

6.11 Applications of Hashing

Hashing is used in dictionaries/maps, symbol tables, caches, database indexing, duplicate detection, compiler tables and many lookup systems.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

6.12 Advantages and Limitations

Advantages: fast average lookup, direct key-based access and flexible mapping.
Limitations: collisions, dependence on hash quality, resizing complexity and potentially poor worst-case performance.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

UNIT 6 – HASHING

6.13 Exam Practice

Explain hashing, hash function, collision, separate chaining, linear probing, quadratic probing, double hashing and load factor. Draw a small hash-table example.

Exam tip: Underline the definition, write key steps in numbered form, and mention time complexity wherever applicable.

IMPORTANT EXAM QUESTION BANK

1. Define data structure. Explain its classification with suitable examples.
2. Explain array insertion and deletion with a suitable example.
3. Explain singly, doubly and circular linked lists with diagrams.
4. Explain stack and its operations. Discuss applications.
5. Convert/evaluate expressions using stack.
6. Explain queue, circular queue, deque and priority queue.
7. Define tree and explain tree terminology and traversals.
8. Explain binary search tree and its operations.
9. Explain graph representation using adjacency matrix and adjacency list.
10. Explain BFS and DFS with an example.
11. Explain linear search and binary search with complexity.
12. Explain bubble, selection and insertion sort.
13. Explain merge sort and quick sort with complexity.
14. Define hashing and explain collision-resolution techniques.
15. Explain load factor and applications of hashing.

Diagram practice: Array, singly linked list, doubly linked list, stack, queue, circular queue, binary tree, BST, heap, graph, adjacency matrix, hash table.

ONE-PAGE COMPLEXITY REVISION

Operation/Algorithm	Typical / Worst-case idea
Array indexed access	O(1)
Array traversal/search	O(n)
Array insertion/deletion (middle)	O(n)
Linked-list traversal/search	O(n)
Linked-list insertion at head	O(1)
Stack push/pop	O(1)
Queue enqueue/dequeue	O(1) with suitable implementation
BST search (balanced/typical)	O(log n); worst O(n)
BFS / DFS	O(V + E) with adjacency lists
Linear Search	O(n) worst case
Binary Search	O(log n) worst case
Bubble Sort	$O(n^2)$ worst case
Selection Sort	$O(n^2)$
Insertion Sort	$O(n^2)$ worst case
Merge Sort	O(n log n)
Quick Sort	O(n log n) average; $O(n^2)$ worst case
Hash table operations	O(1) average; O(n) worst case

Final revision rule: If you can define the topic, draw its structure, explain two operations, give one application and state its complexity, you are ready to write a strong exam answer.